

C++ Programming

1. Introduction to C++

- C++ is a general-purpose, object-oriented programming language developed by Bjarne Stroustrup.
- It is an extension of the C language and supports both procedural and object-oriented programming.
- Important features include classes, objects, inheritance, polymorphism, encapsulation, abstraction, templates, and exception handling.

2. Features of C++

- Object-oriented programming support.
- Fast and efficient execution.
- Supports procedural, object-oriented, and generic programming.
- Provides function overloading and operator overloading.
- Supports inheritance and polymorphism.
- Provides reusable code through classes, functions, and templates

3. Basic Structure of a C++ Program

Header file: Provides library declarations.

- `main()`: Starting point of program execution.

Statements: Instructions executed by the program.

- `return 0`: Indicates successful program termination.

4. Tokens in C++

- Keywords: Reserved words such as int, class, public, private, if, else, return.
- Identifiers: Names given to variables, functions, classes, and objects.
- Constants: Fixed values.
- Operators: Symbols used to perform operations.
- Separators: Symbols such as (), {}, [], ; and ,.

5. Data Types

- Fundamental: int, char, float, double, bool, void.
- Derived: arrays, pointers, functions, references.
- User-defined: class, struct, union, enum.

6. Variables and Constants

- A variable is a named memory location whose value can change during program execution.
- A constant represents a value that should not be changed after initialization.
- Example: `const int MAX = 100;`

7. Operators in C++

- Arithmetic: +, -, *, /, %
- Relational: ==, !=, >, =, <=
- Logical: &&, ||, !

- Assignment: =, +=, -=, *=, /=
- Increment/Decrement: ++, --
- Conditional: ? : • Bitwise: &, |, ^, ~, <>

7. Input and Output

- cout is used to display output.
- cin is used to accept input.
- endl moves the cursor to the next line.
- Example: cout << "Hello"; cin >> age;

8. Decision Making Statements

- if: Executes statements when a condition is true.
- if-else: Selects between two alternatives.
- else-if ladder: Checks multiple conditions.
- switch: Selects one case from multiple choices.

10. Looping Statements

for: Useful when the number of iterations is known

- while: Repeats while a condition is true.
- do-while: Executes the body at least once.

11. Functions

- A function is a block of code designed to perform a specific task

- Functions improve code reuse, modularity, testing, and maintenance.
- Function overloading: Multiple functions can have the same name with different parameter lists.

12. Object-Oriented Programming (OOP)

- Class: A blueprint that defines data members and member functions.
- Object: An instance of a class.
- Encapsulation: Binding data and methods together and controlling access.
- Abstraction: Showing essential details while hiding implementation details.
- Inheritance: Creating a new class from an existing class.
- Polymorphism: One interface/name can represent different behaviors.

13. Access Specifiers

- public: Members can be accessed from outside the class.
- private: Members are accessible only within the class (and permitted friends)
- protected: Members are accessible within the class and derived classes.

14. Constructors and Destructors

- Constructor: Special member function called automatically when an object is created.
- Destructor: Special member function called when an object is destroyed.

- Constructor name is the same as the class name. Destructor starts with ~ followed by the class name.

15. Inheritance

- Single: One base class and one derived class.
- Multiple: A derived class inherits from more than one base class.
- Multilevel: Inheritance occurs in multiple levels. Hierarchical: Multiple derived classes inherit from one base class.
- Hybrid: Combination of two or more inheritance types.

16. Polymorphism

- Compile-time polymorphism: Function overloading and operator overloading.
- Run-time polymorphism: Achieved using virtual functions and overriding.

17. Arrays and Strings

- An array stores multiple elements of the same data type.
- C++ provides the `std::string` class for convenient string handling.
- Example: `int marks[5];`
- Example: `string name = "Anuradha";`

18. Pointers and References

- A pointer stores the memory address of another variable.
- A reference is an alias for an existing variable.

- Pointers use * and address-of operator & is used to obtain an address.

19. Exception Handling

- Exception handling is used to handle run-time errors safely.
- try: Contains code that may generate an exception.